

Redis基础

一、Redis概念

1.简介

Redis是一个由 Salvatore Sanfilippo 写的 key-value 存储系统，是跨平台的非关系型数据库。是一个开源的使用 ANSI C 语言编写、遵守 BSD 协议、支持网络、可基于内存、分布式、可选持久性的键值对(Key-Value)存储数据库，并提供多种语言的 API。通常被称为数据结构服务器，因为值（value）可以是字符串(String)、哈希(Hash)、列表(list)、集合(sets)和有序集合(sorted sets)等类型。

2.优势

- 性能极高 – Redis能读的速度是110000次/s,写的速度是81000次/s。
- Redis支持二进制案例的 Strings, Lists, Hashes, Sets 及 Ordered Sets 数据类型操作。

3.基础知识

redis默认有16个数据库，默认使用第0个数据库，他可以作用为数据库，缓存，消息中间件。

```
[root@localhost redis]# grep 'databases' conf/6379.conf
databases 16
```

redis是单线程的，redis速度非常快，官方表示redis是基于内存操作，redis的瓶颈是根据内存和网络带宽来决定。redis将所有数据存放在内存的，所以使用单线程去操作效率更高。多线程cpu会切换上下文，所以会耗时。

二、Redis安装

Redis 支持 32 位和 64 位。这个需要根据你系统平台的实际情况选择，这里我选择redis-5.0.13.tar.gz

1.下载安装包

```
#下载地址
https://redis.io/download/
#创建文件夹上传安装包
[root@localhost ~]# mkdir -p /opt/soft/redis && cd /opt/soft/redis
# 查看包是否上传成功
[root@localhost /opt/soft/redis]# ll
总用量 1952
-rw-r--r-- 1 root root 1995566 1月 16 09:20 redis-5.0.13.tar.gz
#解压并切换到解压后的目录
[root@localhost /opt/soft/redis]# tar -zxvf redis-5.0.13.tar.gz
[root@localhost /opt/soft/redis]# cd redis-5.0.13
```

2.安装依赖

```
yum -y install gcc-c++
```

3.执行安装命令

```
#创建目录，存放安装后的文件
mkdir /usr/local/redis
#指定目录安装
make PREFIX=/usr/local/redis install
#编译并安装
make && make install
```

4.配置文件拿取

```
#创建目录
mkdir /usr/local/redis/conf
#复制配置文件到/usr/local/redis/conf
cp /opt/soft/redis/redis-5.0.13/redis.conf /usr/local/redis/conf/redis.conf
#查看redis默认配置
[root@localhost ~]# grep -v '^#' /usr/local/redis/conf/redis.conf | grep -v '^$'
bind 127.0.0.1
protected-mode yes
port 6379
tcp-backlog 511
timeout 0
tcp-keepalive 300
daemonize no
supervised no
pidfile /var/run/redis_6379.pid
loglevel notice
logfile ""
databases 16
always-show-logo yes
save 900 1
save 300 10
save 60 10000
stop-writes-on-bgsave-error yes
rdbcompression yes
rdbchecksum yes
dbfilename dump.rdb
dir ./
replica-serve-stale-data yes
replica-read-only yes
repl-diskless-sync no
repl-diskless-sync-delay 5
repl-disable-tcp-nodelay no
replica-priority 100
lazyfree-lazy-eviction no
lazyfree-lazy-expire no
lazyfree-lazy-server-del no
replica-lazy-flush no
appendonly no
appendfilename "appendonly.aof"
appendfsync everysec
no-appendfsync-on-rewrite no
auto-aof-rewrite-percentage 100
```

```
auto-aof-rewrite-min-size 64mb
aof-load-truncated yes
aof-use-rdb-preamble yes
lua-time-limit 5000
slowlog-log-slower-than 10000
slowlog-max-len 128
latency-monitor-threshold 0
notify-keyspace-events ""
hash-max-ziplist-entries 512
hash-max-ziplist-value 64
list-max-ziplist-size -2
list-compress-depth 0
set-max-intset-entries 512
zset-max-ziplist-entries 128
zset-max-ziplist-value 64
hll-sparse-max-bytes 3000
stream-node-max-bytes 4096
stream-node-max-entries 100
activeremhashing yes
client-output-buffer-limit normal 0 0 0
client-output-buffer-limit replica 256mb 64mb 60
client-output-buffer-limit pubsub 32mb 8mb 60
hz 10
dynamic-hz yes
aof-rewrite-incremental-fsync yes
rdb-save-incremental-fsync yes
```

修改配置文件

```
[root@localhost ~]# vim /usr/local/redis/conf/redis.conf
bind 0.0.0.0 #修改后才能让本机所有ip都可以进行登录
protected-mode yes
port 6379 #端口号
tcp-backlog 511
timeout 0
tcp-keepalive 300
daemonize yes #是否后台运行，默认是no，需要改成yes
supervised no
pidfile /var/run/redis_6379.pid
loglevel notice
logfile ""
databases 16
always-show-logo yes
save 900 1
save 300 10
save 60 10000
stop-writes-on-bgsave-error yes
rdbcompression yes
rdbchecksum yes
dbfilename dump.rdb
dir ./ #数据持久化保存路径
replica-serve-stale-data yes
replica-read-only yes
```

```
repl-diskless-sync no
repl-diskless-sync-delay 5
repl-disable-tcp-nodelay no
replica-priority 100
lazyfree-lazy-eviction no
lazyfree-lazy-expire no
lazyfree-lazy-server-del no
replica-lazy-flush no
appendonly no
appendfilename "appendonly.aof"
appendfsync everysec
no-appendfsync-on-rewrite no
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
aof-load-truncated yes
aof-use-rdb-preamble yes
lua-time-limit 5000
slowlog-log-slower-than 10000
slowlog-max-len 128
latency-monitor-threshold 0
notify-keyspace-events ""
hash-max-ziplist-entries 512
hash-max-ziplist-value 64
list-max-ziplist-size -2
list-compress-depth 0
set-max-intset-entries 512
zset-max-ziplist-entries 128
zset-max-ziplist-value 64
hll-sparse-max-bytes 3000
stream-node-max-bytes 4096
stream-node-max-entries 100
activerehashing yes
client-output-buffer-limit normal 0 0 0
client-output-buffer-limit replica 256mb 64mb 60
client-output-buffer-limit pubsub 32mb 8mb 60
hz 10
dynamic-hz yes
aof-rewrite-incremental-fsync yes
rdb-save-incremental-fsync yes
```

启动redis

```
/usr/local/redis/bin/redis-server /usr/local/redis/conf/redis.conf
```

5.加入systemctl管理

可以加可以不加，不加就按照上面的方式进行启动

```

cp /opt/soft/redis/redis-5.0.13/utils/redis_init_script /etc/init.d/redis
#修改启动文件的配置文件路径
sed -ri 's#EXEC=/usr/local/bin/redis-server#EXEC=/usr/local/redis/bin/redis-server#g'
/etc/init.d/redis
sed -ri 's#CLIEEXEC=/usr/local/bin/redis-cli#CLIEEXEC=/usr/local/redis/bin/redis-cli#g'
/etc/init.d/redis
sudo sed -ri 's#CONF="/etc/redis/\${
{REDISPORT}\}.conf"#CONF="/usr/local/redis/conf/redis.conf"#' /etc/init.d/redis
#授权
chmod +x /etc/init.d/redis
chkconfig --add redis
chkconfig redis on
#重新装载
systemctl daemon-reload
#启动, 如果之前有启动, 需要先停止
systemctl start redis

```

6.Redis工具

```

./redis-benchmark    #用于进行redis性能测试工具
./redis-check-dump    #用于修复备份数据文件
./redis-cli          #redis的客户端
./redis-server       #redis服务端
./redis-check-aof    #修复出问题的AOF文件
./redis-sentinel     #用于集群管理

```

7.测试

```

[root@localhost ~]# /usr/local/redis/bin/redis-cli
127.0.0.1:6379> ping    #测试是否连接成功
PONG    #代表连接成功
127.0.0.1:6379> set name heber    #设置一个key并赋值
OK
127.0.0.1:6379> get name    #获取一个key
"heber"
127.0.0.1:6379>quit #退出登录

```

8.redis-benchmark压力测试

```

#测试100个并发, 100000个请求
[root@localhost ~]# /usr/local/redis/bin/redis-benchmark -h 127.0.0.1 -p 6379 -c 100 -n 100000

```

三、基本命令

1.select切换数据库

```

127.0.0.1:6379> select 1    #切换到1号库
OK
127.0.0.1:6379[1]>

```

2.查看数据库空间

```
127.0.0.1:6379> dbsize
(integer) 0 #0表示没有
127.0.0.1:6379> set name heber
OK
127.0.0.1:6379> dbsize
(integer) 1 #1表示存在一个
127.0.0.1:6379>
```

3.查看所有的key

```
127.0.0.1:6379[1]> keys *
1) "name"
127.0.0.1:6379[1]>
```

4.插入数据

```
127.0.0.1:6379> set name heber
OK
```

5.获取数据

```
127.0.0.1:6379> get name
"heber"
```

4.清空所有库

```
flushall
```

5.清空当前库

```
flushdb
```

6.判断键是否存在

```
127.0.0.1:6379> exists name
(integer) 1 #1代表存在
```

7.移除键

```
127.0.0.1:6379> keys *
1) "name"
2) "mylist"
127.0.0.1:6379> move mylist 2
(integer) 1 #1代表移除一个成功
```

8.设置过期时间

```
127.0.0.1:6379> expire name 10 #10代表10秒
(integer) 1
127.0.0.1:6379> ttl name #查看过期时间
(integer) 2 #还有2秒过期
127.0.0.1:6379> ttl name
(integer) -2 #-2代表已过期
```

9.查看key的数据类型

```
127.0.0.1:6379> type name
string #数据类型为string
```

10.查看当前库的信息

```
127.0.0.1:6379> info replication
# Replication
role:master #当前角色
connected_slaves:0 #从机
```

四、redis数据类型

1.redis五大数据类型

1.string(字符串)

2.list(列表)

3.set(集合)

4.hash(散列)

5.zset(有序集合)

2.三种特殊数据类型

1.geospatial

2.hyperloglog

3.bitmaps

五、配置文件

1.redis.conf

```
#引入其他文件
include /path/to/other.conf
#加载一些so文件
loadmodule /path/to/other_module.so
#网络配置，绑定自己的ip
bind 127.0.0.1
#是否受保护，默认打开
protected-mode yes
#设置端口号
```

```
port 6379
#tcp连接
tcp-backlog 511
timeout 0
tcp-keepalive 300
#是否守护进程开启,默认是no
daemonize yes
#管理守护进程
supervised no
#配置pid, 如果以后台形式运行就需要指定pid
pidfile /var/run/redis_6379.pid
#设置日志级别
loglevel notice
#日志生成的文件位置名
logfile ""
#默认数据库数量
databases 16
#是否显示logo
always-show-logo yes
#快照, 在指定时间内执行了多少次操作, 会写入持久化文件.rdb
save 900 1
save 300 10
save 60 10000
#持久化出错后是否继续工作
stop-writes-on-bgsave-error yes
#是否压缩rdb文件
rdbcompression yes
#是否校验, 如果出错自动修复
rdbchecksum yes
#
dbfilename dump.rdb
#rdb保存目录
dir ./
#主从复制
replica-serve-stale-data yes
replica-read-only yes
repl-diskless-sync no
repl-diskless-sync-delay 5
repl-disable-tcp-nodelay no
replica-priority 100
#设置密码
requirepass foobared
#客户的限制
maxclients 10000
#内存设置
maxmemory <bytes>
#是否开启aof
appendonly no
#持久化名
appendfilename "appendonly.aof"
#每秒执行一次
appendfsync everysec
no-appendfsync-on-rewrite no
```



```
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
aof-load-truncated yes
aof-use-rdb-preamble yes
```

六、Redis持久化

redis是内存数据库，如果不将内存中的数据状态保存在磁盘，那么一旦退出进程，数据库中数据也会消失，所以redis提供了持久化。开启持久化后，重启redis后，数据会自动通过持久化文件恢复。

redis的持久化有两种：

RDB rdb是在不同的时间点，将redis存储的数据生成快照存储到磁盘等介质上。优缺点：周期性的，不影响数据写入，高效，完整性差。RDB默认开着 AOF aof是将redis执行过程中所有指令记录在日志中，在下次redis重新启动时，只要把这些命令全部重新执行一下就可以实现数据恢复。优缺点：实时性，完整性好，体积大。默认关闭

1.RDB

rdb默认是打开的，通过配置文件可以查看备份的频率，达到频率才会备份

```
vim /usr/local/redis/conf/redis.conf
#备份的频率
save 900 1 #15分钟最少有几个key改变
save 300 10 #5分钟最少有10key改变
save 60 10000 #1分钟最少有1w个key改变
#备份的文件名称
dbfilename dump.rdb
#数据持久化保存路径，没有需要创建
dir /usr/local/redis/data
# 启用压缩（节省空间，轻微性能损耗）
rdbcompression yes
# 写入校验和（数据完整性验证）
rdbchecksum yes
# 后台保存失败时禁止写入（保障数据安全）
stop-writes-on-bgsave-error yes
```

手动进行备份

```
127.0.0.1:6379> save
```

数据还原需要先停掉redis服务。然后将备份文件存放在备份目录中即可

2.AOF

```
vim /usr/local/redis/conf/redis.conf
#由于AOF默认关闭所以需要打开
appendonly yes
#备份文件名
appendfilename "appendonly.aof"
# 与 RDB 共用目录，需确保权限正确
dir /usr/local/redis/data
# 推荐值，平衡性能与安全性
appendfsync everysec
```

数据还原需要先停掉redis服务。然后将备份文件存放在备份目录中即可

七、Redis主从

1.准备好3台服务器并安装好redis

```
#这里的环境
192.168.47.130 #master
192.168.47.141 #slave
192.168.47.142 #slave
```

2.修改主服务器配置文件

```
vim /usr/local/redis/conf/redis/6379.conf
bind 0.0.0.0
#关闭保护模式
protected-mode no
```

3.修改从服务器配置文件

```
vim /etc/redis/6379.conf
#5版本使用这个
replicaof 192.168.47.130 6379
#5以下使用这个
slaveof 192.168.47.130 6379
bind 0.0.0.0
protected-mode no
```

4.验证

```
#查看主从是否正常
127.0.0.1:6379> info replication
# Replication
role:master
connected_slaves:2 #这里显示两台从服务下面显示分别是141和142
slave0:ip=192.168.47.142,port=6379,state=online,offset=168,lag=1
slave1:ip=192.168.47.141,port=6379,state=online,offset=168,lag=1
master_replid:7eb964b0eee9b0cb08b50a2f751449b33d2ca211
master_replid2:0000000000000000000000000000000000000000
master_repl_offset:168
second_repl_offset:-1
```

```
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:1
repl_backlog_histlen:168
#从服务只是用get name
[root@localhost ~]# /usr/local/redis/bin/redis-cli
127.0.0.1:6379> set name heber
OK
127.0.0.1:6379> get name
"heber"
```

八、Redis哨兵sentinel

Redis 哨兵是 Redis 官方提供的高可用性解决方案，用于监控、管理和自动修复 Redis 主从架构中的故障，解决单点故障问题。它的核心目标是实现 Redis 主从架构的自动故障转移（Failover），确保服务持续可用。

1.每台机器上修改主配置文件

```
bind 0.0.0.0
```

2.每台机器上修改sentinel配置文件

```
#复制哨兵配置文件，从源码包中复制
cp /opt/soft/redis/redis-5.0.13/sentinel.conf /usr/local/redis/conf/sentinel.conf
#编辑配置文件
vim /usr/local/redis/conf/sentinel.conf
port 26379
#主服务器ip
sentinel monitor mymaster 192.168.47.130 6379 2
sentinel down-after-milliseconds mymaster 3000
sentinel failover-timeout mymaster 10000
```

从服务器中添加

```
vim /usr/local/redis/conf/sentinel.conf
```

3.每台机器启动哨兵

```
/usr/local/redis/bin/redis-sentinel /usr/local/redis/conf/sentinel.conf
```

九、Redis集群

Redis 集群是 Redis 官方提供的分布式解决方案，通过将数据分片（Sharding）存储在多个节点上，实现高可用性、横向扩展和负载均衡。它解决了单机 Redis 在内存容量、性能和高可用性上的限制。

1.环境规划

3主3从

编号	主机名称	ip地址	角色
1	redis1	192.168.0.130	redis7001
2	redis1	192.168.0.130	redis7002
3	redis2	192.168.0.131	redis7003
4	redis2	192.168.0.131	redis7004
5	redis3	192.168.0.132	redis7005
6	redis3	192.168.0.132	redis7006

2.部署redis

参考前面的安装即可，但是配置文件名称有变，在/usr/local/redis/conf/下创建以下文件

redis1 => redis7001.conf/redis7002.conf

redis2 => redis7003.conf/redis7004.conf

redis3 => redis7005.conf/redis7006.conf

配置文件

```
bind 0.0.0.0 #修改后才能让本机所有ip都可以进行登录
port 7001 #端口号，对应每一个配置文件
cluster-enabled yes #开启集群
cluster-config-file nodes_7001.conf
cluster-node-timeout 5000
daemonize yes #是否后台运行，默认是no，需要改成yes
dbfilename dump.rdb #rdb持久化
dir /usr/local/redis/data #数据持久化保存路径
pidfile /var/run/redis_7001.pid #进程id
appendonly yes #aof持久化
appendfilename "appendonly.aof"
appendfsync everysec
protected-mode no #关闭保护模式
tcp-backlog 511
timeout 0
tcp-keepalive 300
supervised no
loglevel notice
logfile ""
databases 16
always-show-logo yes
save 900 1
save 300 10
save 60 10000
stop-writes-on-bgsave-error yes
rdbcompression yes
rdbchecksum yes
replica-serve-stale-data yes
replica-read-only yes
```

```
repl-diskless-sync no
repl-diskless-sync-delay 5
repl-disable-tcp-nodelay no
replica-priority 100
lazyfree-lazy-eviction no
lazyfree-lazy-expire no
lazyfree-lazy-server-del no
replica-lazy-flush no
no-appendfsync-on-rewrite no
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
aof-load-truncated yes
aof-use-rdb-preamble yes
lua-time-limit 5000
slowlog-log-slower-than 10000
slowlog-max-len 128
latency-monitor-threshold 0
notify-keyspace-events ""
hash-max-ziplist-entries 512
hash-max-ziplist-value 64
list-max-ziplist-size -2
list-compress-depth 0
set-max-intset-entries 512
zset-max-ziplist-entries 128
zset-max-ziplist-value 64
hll-sparse-max-bytes 3000
stream-node-max-bytes 4096
stream-node-max-entries 100
activeremhashing yes
client-output-buffer-limit normal 0 0 0
client-output-buffer-limit replica 256mb 64mb 60
client-output-buffer-limit pubsub 32mb 8mb 60
hz 10
dynamic-hz yes
aof-rewrite-incremental-fsync yes
rdb-save-incremental-fsync yes
```

启动这6个节点的redis

```
/usr/local/redis/bin/redis-server /usr/local/redis/conf/redis700x.conf
```

3.启动集群

```
/usr/local/redis/bin/redis-cli --cluster create 192.168.0.130:7001 192.168.0.131:7004
192.168.0.132:7006 192.168.0.132:7005 192.168.0.131:7003 192.168.0.130:7002 --cluster-
replicas 1
```

4.测试

```
/usr/local/redis/bin/redis-cli -c -h 主节点ip -p 7001
set name heber
```

5.关闭集群

```
/usr/local/redis/bin/redis-cli -c -h 192.168.0.130 -p 7001 shutdown  
/usr/local/redis/bin/redis-cli -c -h 192.168.0.130 -p 7002 shutdown  
/usr/local/redis/bin/redis-cli -c -h 192.168.0.131 -p 7003 shutdown  
/usr/local/redis/bin/redis-cli -c -h 192.168.0.131 -p 7004 shutdown  
/usr/local/redis/bin/redis-cli -c -h 192.168.0.133 -p 7005 shutdown  
/usr/local/redis/bin/redis-cli -c -h 192.168.0.133 -p 7006 shutdown
```

6.重启集群

```
#依次启动即可  
/usr/local/redis/bin/redis-server /usr/local/redis/conf/redis700x.conf
```